

TestNG

Automação, organização e execução de testes em Java

Leonardo Freitas

Segurança e Teste de Software • IDP

O foco da apresentação

Como o TestNG ajuda quando a suíte cresce: dados, grupos, dependências e paralelismo.

O problema não é escrever 1 teste. É organizar 1.000.

Uma suíte grande precisa de regras claras de execução.

1. Repetição

O mesmo cenário precisa ser executado com muitos dados.

2. Organização

Nem sempre queremos rodar todos os testes.

3. Escala

A execução pode precisar acontecer em paralelo.

TestNG entra como framework/test runner para coordenar essa execução.

O que é TestNG?

Testing Next Generation — framework de testes para Java.

Test runner

Descobre e executa métodos marcados como teste.

Lifecycle

Prepara e limpa o ambiente dos testes.

Data-driven

Executa o mesmo teste com vários dados.

Orquestração

Groups, dependencies e paralelismo.

TestNG não é Selenium e não é uma técnica de teste.

Ele organiza COMO os testes serão executados.

Onde o TestNG pode ser usado?

Ele pode coordenar testes em diferentes

UI / E2E

TestNG + Selenium / Appium

Serviço / API / Integração

TestNG + REST Assured / clientes HTTP

Unidade

Java + TestNG

Nosso sistema: TransferService

Um exemplo simples para concentrar a atenção no framework.

Regra 1

Transferência mínima: R\$ 10,00

Regra 2

Transferência máxima: R\$
10.000,00

Regra 3

O valor não pode ultrapassar o
saldo.

Sistema simples → testes fáceis de entender → recursos do TestNG ficam visíveis.

@TEST

A anatomia de um teste simples

O TestNG encontra o método, executa e verifica a assertção.

```
@Test(groups = {"smoke", "regression"})
public void shouldAcceptAValidTransfer() {
    boolean result = service.get().isValid(5000, 1000);
    Assert.assertTrue(result);
}
```

@Test = “este método é um caso de teste”.

@DataProvider + Boundary Value Analysis

A técnica escolhe os valores. O TestNG fornece os dados ao teste.

9 inválido	10 válido	11 válido	9999 válido	10000 válido	10001 inválido
----------------------	---------------------	---------------------	-----------------------	------------------------	--------------------------

BVA testa a vizinhança das fronteiras: abaixo, no limite e acima.

Limites usados: 10 e 10.000.

ECP — Particionamento de Equivalência

Em vez de testar infinitos valores, escolhemos representantes de classes.

Abaixo do mínimo

-50, 5 → inválidos

Classe válida

500, 5.000 → válidos

Acima do máximo

15.000 → inválido

ECP escolhe O QUE testar. TestNG organiza COMO executar.

Groups: execute apenas o que interessa

Os testes podem pertencer a diferentes conjuntos.

smoke

regression

bva

ecp

workflow

```
mvn -Dgroups=smoke test
```

Útil em CI/CD: smoke em commits rápidos; regressão em execuções mais completas.

Dependencies: um teste pode depender de outro

TestNG possui dependências declarativas entre métodos.



```
@Test(dependsOnMethods = "authenticate")
```

Cuidado: dependências demais aumentam o acoplamento entre testes.

Paralelismo: três métodos ao mesmo tempo

O testng.xml controla a estratégia de execução.

```
<suite  
  parallel="methods"  
  thread-count="3">
```

Thread safety

Cada thread precisa trabalhar com estado isolado.

No projeto usamos `ThreadLocal<TransferService>` para evitar interferência entre threads.

Paralelismo sem isolamento pode criar race conditions.

Demonstração ao vivo — sequência de 5 minutos

A ideia é mostrar poucos pontos, mas todos funcionando.

1.
TransferService
regras

2.
TransferServiceTest
@Test + DataProvider

3.
TransferWorkflowTest
dependencies

4. **testng.xml**
parallel + threads

```
mvn test  
mvn -Psuite test
```

TestNG x JUnit moderno

Recurso	TestNG	JUnit
Testes unitários	✓	✓
Parametrização	@DataProvider	@ParameterizedTest
Agrupamento	groups	@Tag
Paralelismo	✓	✓
Dependências	nativo	sem equivalente direto
Suite XML	forte	abordagem diferente

Vantagens e cuidados

Recursos fortes também criam responsabilidades.

Vantagens

- DataProvider • Groups • Dependencies • Suite XML • Paralelismo

Cuidados

- Configuração adicional • Testes podem ficar acoplados
- Paralelismo exige thread safety • Há sobreposição com JUnit

O que este estudo de caso demonstrou

TestNG ganha valor quando a suíte precisa de organização e escala.

Técnica

BVA + ECP definem casos importantes.

Framework

TestNG executa, agrupa e parametriza.

Escala

Dependencies + parallel organizam a suíte.

BVA e ECP dizem O QUE testar. TestNG ajuda a decidir COMO executar.

PERGUNTAS

Obrigado.

Leonardo Freitas • TestNG 7.12.0

Referências

testng.org/documentation.html • github.com/testng-team/testng • docs.junit.org • Apache Atlas • Apache BifroMQ